

JustDial Scraper

Complete Process Plan & Technical Blueprint

Version 1.0 | Data Collection & Automation | For Approval

About This Document

This document is the complete process plan for building a JustDial scraper. It explains — in plain language — how the site works, how data is extracted, how phone numbers are decoded, how records are stored in CSV, and how every failure scenario is handled gracefully.

No code is included here. This is a planning blueprint. Implementation begins only after this document is reviewed and approved.

Step 1

Understanding How JustDial Works

Before start, we study the target site thoroughly. The goal is to understand every mechanism JustDial uses — so we can work around each one intelligently.

Manual Exploration — What to Do

1. Open justdial.com in your browser
2. Search for "AC Repair" in location 768028
3. Watch the URL change and observe how listing cards appear
4. Right-click the page and View Page Source — look at the phone number HTML
5. Scroll down slowly and notice new cards loading below the fold
6. Click "Get Number" on one listing and watch the Network tab in Developer Tools

What You Will Discover

Observation	What It Means for Our Scraper
URL becomes /768028/ac-repair	URL structure is predictable — we can build it programmatically
Phone number shows as random letters	JustDial encodes phone numbers — we must decode them

Scrolling triggers more cards to load	Lazy loading is active — we must scroll to get all results on a page
"Get Number" button triggers a network call	Phone can also be retrieved by clicking the button as a fallback
Bottom of page has a "Next Page" button	Pagination exists — we must loop through multiple pages

Key Insight: JustDial renders content dynamically using JavaScript. A plain HTTP request will only return an empty skeleton. We must use a real browser engine that can execute scripts, scroll the page, and interact with elements — exactly like a human would.

Step 2

The Phone Number Problem — Cipher Decoding

This is the single most important challenge in the entire scraper. JustDial never writes real phone digits in their HTML. They use a substitution cipher — each digit is replaced with a letter.

The Cipher Map

Real Digit	0	1	2	3	4	5	6	7	8	9
Encoded As	c	d	e	f	g	h	i	j	k	l

Example: The phone number 9876543210 is written in JustDial's HTML as the text lkjihgfedcc. We read those letters and translate them back to digits using the map above.

Two Methods to Get the Real Number

Method	How It Works	Speed	When to Use
Option A — Cipher Decode	Read encoded text from HTML, apply the cipher map character by character, get the real number instantly	Fast — no clicks needed	Primary method — used first for every listing
Option B — Click Button	Physically click the "Get Number" button, wait for the network response, read the number from the popup	Slow — one network call per listing	Fallback — used only if Option A produces garbage output

Our Strategy

Always try Option A first. If the decoded result is not a valid 10-digit number, automatically

fall back to Option B. This gives us speed as the default and accuracy as the safety net.

Step 3 URL Pattern & Navigation

JustDial uses a completely predictable URL structure. Once we understand it, navigating to any page of results is trivial.

URL Structure

```
Page 1: justdial.com/768028/ac-repair
Page 2: justdial.com/768028/ac-repair/page-2
Page 3: justdial.com/768028/ac-repair/page-3
```

How We Build the URL Automatically

7. User enters a skill name, e.g. "AC Repair"
8. We convert it to lowercase and replace spaces with hyphens: ac-repair
9. We insert the pincode and formatted skill into the URL template
10. For each subsequent page, we simply increment the page number

Step 4 Data Fields Extracted Per Listing

Each business card on JustDial contains multiple data points. Here is every field we extract, and exactly where it is found within the HTML of each card:

Field	Example Value	Location in HTML
Name	Cool Point AC Services	Heading or span — the largest text element inside the card
Phone	9876543210	Span with encoded letters — decoded using the cipher map
Address	Shop 12, MG Road, Bhubaneswar	Smaller text block below the business name
Rating	4.2	Coloured box showing a decimal number out of 5
Review Count	38	Text like "38 Ratings" sitting next to the rating box
Category	AC Repair & Service	Tag or label displayed below the business name
Email	(blank)	Almost never shown on free listings — always stored as blank

Step 5 Browser Automation — Action by Action

Our script controls a real Chrome browser programmatically. Think of it as a robot sitting at a keyboard, following a precise sequence of actions for every page it visits.

Exact Sequence of Actions

#	Action Taken	Why It Is Necessary
1	Open a Chrome browser window	JustDial requires JavaScript — plain HTTP requests return an empty page
2	Navigate to the constructed URL for page 1	Loads the first page of search results
3	Wait until business card elements are visible	The page loads in stages — cards appear after the initial HTML skeleton
4	Scroll down slowly, pause, scroll again — repeat 4 times	Lazy loading only reveals cards when they enter the visible viewport
5	Capture the complete HTML of the current page	Gives the parser everything it needs to extract all business data
6	Pass the HTML to the parser module	The parser identifies cards and extracts the 8 data fields from each one
7	Wait a random 2 to 5 seconds	Mimics human browsing behaviour — reduces the chance of JustDial blocking us
8	Navigate to page 2 and repeat from action 3	Paginates through all available result pages
9	Stop when zero results are returned or page limit is reached	Prevents wasting time on empty pages or hitting anti-bot measures
10	Close the browser window	Releases memory and system resources cleanly after the run

Step 6 How the HTML Parser Works

Once the browser captures a page's full HTML, the parser takes over. It works on plain text — no browser interaction needed at this point.

Parser Logic — Step by Step

11. Locate all business card containers on the page (typically 10 to 15 cards per page)
12. For each card: find the name element and extract its text

13. Find the phone element, extract the encoded text, apply the cipher map to decode it
14. Validate the decoded phone — it must be exactly 10 digits. If not, flag it and try Option B
15. Find the address element and extract its text
16. Find the rating element and extract the decimal number
17. Find the review count and extract the integer
18. Find the category tag and extract its text
19. Bundle all 8 fields into one record — one row of output data
20. Move to the next card and repeat until all cards on the page are processed

Missing Field Policy

If any field is not found on a card, it is stored as blank. We never skip an entire record because one field is missing. A partial record is always better than no record.

Step 7

Pagination Logic

After parsing each page, the scraper decides whether to continue to the next page using a simple two-condition rule:

Condition	Action
Current page returned 1 or more results	Move to next page and continue scraping
Current page returned 0 results	Stop — there are no more listings to collect
Maximum page limit reached (default: 5 pages)	Stop — safety limit prevents running forever on large searches

Example Pagination Run

```
Page 1 --> 15 results --> continue to page 2
Page 2 --> 15 results --> continue to page 3
Page 3 --> 8 results --> continue to page 4
Page 4 --> 0 results --> STOP. Save all 38 records.
```

Why 5 pages? Without a limit, a broad search like "electrician" in a large city could return 100+ pages, causing the scraper to run for hours. 5 pages yields approximately 50 to 75 results — enough for most practical use cases. The limit is adjustable at runtime.

Step 8

Data Cleaning & Deduplication

Raw scraped data is always messy. Before any record is saved, every field goes through a cleaning and deduplication process.

Cleaning Steps Applied to Every Record

- Strip leading and trailing spaces and newline characters from all text fields
- Remove non-digit characters from phone numbers — brackets, dashes, spaces, dots
- If the cleaned phone number is fewer than 7 digits, mark it as blank (clearly invalid)
- Normalise the rating to a plain decimal number — remove stars, units, and extra text
- Trim address and category fields to remove any HTML artefacts that crept in

Deduplication — Detecting Repeat Listings

The same business sometimes appears on multiple pages, or with slightly different name spellings. We use a combined key to detect and remove duplicates:

Duplicate Key = lowercase(name) + first 6 digits of phone

If two records produce the same key, we keep the first occurrence and discard the second. This catches both exact duplicates and listings with minor name variations (e.g. "Cool Point AC" vs "Cool Point A.C.").

Step 9

Data Storage — Complete CSV Guide

All extracted, cleaned, and deduplicated records are saved to a CSV file. This section explains — in full detail — what CSV is, how the file is structured, how it is created, and how it handles being run multiple times.

What is a CSV File?

CSV stands for Comma-Separated Values. It is a plain text file where each line represents one business, and each piece of data (name, phone, address, etc.) is separated by a comma. CSV files have no proprietary format — they can be opened in Microsoft Excel, Google Sheets, LibreOffice Calc, or any text editor without any special software.

Why CSV and Not a Database?

Reason	Explanation
Universal compatibility	Works in any spreadsheet app or text editor — no database server, no drivers, no setup

Human-readable	Can be opened in Notepad in an emergency — no special knowledge needed
Easy to share	One file, one email attachment — no export process required
Append-safe	New records can be added to an existing file without overwriting old data
Lightweight	Thousands of records amount to only a few kilobytes of storage
Script-friendly	Every programming language can read and write CSV with built-in tools

File Naming Convention

The output file name is generated automatically from the user's input, so every search gets its own dedicated file:

```
Format:    <skill_with_underscores>_<pincode>.csv
Example:   ac_repair_768028.csv
Location:  output/ac_repair_768028.csv
```

If the user searches for "Plumbing" in pincode 110001, the file would be named plumbing_110001.csv — clearly identifiable at a glance.

CSV Column Structure — All 8 Fields

Every CSV file produced by the scraper has exactly 8 columns, always in this order:

Column	Example Value	Notes
name	Cool Point AC Services	Business name exactly as shown on JustDial
phone	9876543210	10-digit number after cipher decoding and cleaning
email		Email address — left blank for free listings (almost never shown)
address	Shop 12, MG Road, Bhubaneswar	Full address string as listed
rating	4.2	Rating out of 5, stored as a decimal
reviews	38	Number of customer ratings, stored as an integer
category	AC Repair & Service	Service category tag from the listing
source_url	justdial.com/768028/ac-repair	The page URL where this listing was found

How the CSV File is Created — Every Step

21. Scraper finishes all pages for the given pincode and skill
22. All raw records from every page are pooled into one combined list
23. Cleaning is applied to every field in every record
24. Deduplication is applied: duplicates are removed using the name + phone key
25. Scraper checks: does the output file already exist on disk?
26. If NO — create a new file, write the 8-column header row first, then write all data rows one by one
27. If YES — load the existing file into memory, merge new records with old ones, deduplicate the combined set, then rewrite the full file
28. File is written to disk and closed cleanly to prevent data corruption
29. Terminal prints a summary: "47 records saved to output/ac_repair_768028.csv"

Append Mode — Protecting Existing Data

If you run the scraper twice for the same pincode and skill — for example, to refresh results a week later — the second run never deletes the first run's data. Here is exactly what happens in append mode:

- The existing CSV file is loaded into memory as a list of records
- New records from the current run are added to that in-memory list
- The combined list is deduplicated (so no record appears twice from re-scraping the same listing)
- The full merged and deduplicated list is written back to the same file

You can run the scraper as many times as you want. Data is always accumulated, never overwritten. Each run adds genuinely new records and silently drops duplicates.

What the CSV File Looks Like

When opened in a plain text editor, the first few lines of the file look like this:

```
name,phone,email,address,rating,reviews,category,source_url
Cool Point AC Services,9876543210,,Shop 12 MG Road,4.2,38,AC Repair &
Service,justdial.com/768028/ac-repair
FreezFast Solutions,8765432109,,Plot 7 Nayapalli,3.8,12,Air Conditioner
Repair,justdial.com/768028/ac-repair
Arctic Air Tech,9988776655,,Unit 3 Saheed Nagar,4.5,61,AC Service &
Repair,justdial.com/768028/ac-repair
```

When opened in Microsoft Excel or Google Sheets, each comma-separated value appears in its own cell, making the data easy to sort, filter, and share with any team member — no technical knowledge required.

Step 10

Handling Failures Gracefully

Web scraping is unpredictable. JustDial may change its structure, block requests, or simply time out. Our scraper is built to handle every known failure without crashing — it always saves what it has collected.

Problem	What the Scraper Does	Outcome for the User
JustDial blocks the browser	Wait longer between requests; retry once with a different browser identity	Most temporary blocks clear on the first retry
A page loads but shows no cards	Log it, stop pagination for this search, save all records collected so far	Partial results saved — nothing is lost
Phone cipher produces gibberish	Flag the record; store the raw encoded text for manual inspection later	Record is preserved with a note — not silently dropped
Network request times out	Retry the page once automatically; if it fails again, skip and continue to next page	One bad page does not stop the entire run
CAPTCHA screen appears	Stop immediately and alert the user — CAPTCHAs cannot be auto-solved	User is informed; all data collected so far is saved to disk
A single field is missing from a card	Store that field as blank; keep all other fields intact	Partial records are saved, never discarded

Design Principle

The scraper never crashes. Every failure is caught, logged, and handled without stopping the rest of the run. The program always saves whatever it has collected before exiting — even in the worst case scenario.

End-to-End Execution Flow

Complete visual walkthrough of a single scraper run, start to finish

The table below shows every stage of a single run — in exact order — along with what happens at each stage and what the output is:

Stage	What Happens	Output / Result
User Input	User runs: <code>python main.py --pincode 768028 --skill "ac repair"</code>	Pincode and skill are passed to the program
Build URL	Skill converted to ac-repair. URL built: <code>justdial.com/768028/ac-repair</code>	Page 1 URL ready
Open Browser	Chrome launches and navigates to the constructed URL	Browser showing page 1 of results
Wait for Cards	Script waits until business card elements are detected on screen	Page fully loaded and cards visible
Scroll Page	Script scrolls down 4 times with pauses between each scroll	All lazy-loaded cards now visible on the page
Capture HTML	Full HTML of the current page is saved to a variable	Raw HTML string ready for parsing
Parse Cards	Parser finds all card containers, extracts all 8 fields from each one	List of raw records for this page
Decode Phones	Cipher map applied to each encoded phone. Result validated as 10 digits	Real phone numbers in each record
Results Found?	If 0 results: stop. If results found: random 2-5 second delay, go to next page	Either moves to next page or ends the loop
All Pages Done	Records from all pages are merged into one combined list	Single pool of all collected records
Clean & Dedup	Whitespace stripped, phones normalised, ratings cleaned, duplicates removed	Final clean list of unique records
Save to CSV	Records written to <code>output/ac_repair_768028.csv</code> (create or append mode)	CSV file on disk, ready to open
Done	Terminal prints: "47 records saved to <code>output/ac_repair_768028.csv</code> "	Run complete — user can open the file

Ready for Implementation

This document covers every decision, every logic path, and every failure case. Once approved, the code implementation follows this blueprint exactly — step by step.